

Weird Stuff In Traffic

Thomas Cansfield, Hannah Simson, Ludwig Gallmeier, Ahmed Ibrahim, Albert Didkovski, Andreas Kolbinger, Anja Schlaak, Benjamin Kass, Daniel Shaquille, Hamza Dursun, Kevin Kuhn, Martin Lauff, Selin Durmus, Surkhay Khanmammadli, Syed Abidi, Tobias Kerner, Vanessa Rieger

Prof. Dr. Torsten Schön
Technische Hochschule Ingolstadt
Ingolstadt, Germany
torsten.schoen@thi.de



Figure 1. A man made entirely of spaghetti is riding a bike down the middle of the street, 2025.

Abstract

This paper presents a gamified, web-based application designed to enhance the training of object detection models for autonomous driving systems. In the game, users are encouraged to create unusual or "weird" traffic scenes through textual prompts. A fine-tuned AI model then attempts to identify unusual objects within the generated images. The core objective is to identify gaps in the model's recognition capabilities by encouraging users to create scenes that the AI fails to detect correctly. When the AI fails, users are rewarded

with points. Additionally, the system tracks the AI's progress over time, providing feedback on how its recognition performance improves with each iteration. This dual-feedback mechanism—player score and AI progress—aims to create a meaningful engagement loop, motivating users to iteratively contribute to the development of safer autonomous systems. The project highlights the intersection of user experience design, gamification, and machine learning in the context of participatory AI training.

CCS Concepts: • Computing methodologies → Artificial intelligence.

Keywords: Gamification, Object Detection, Generative AI, Inpainting, Human-AI Interaction, Traffic Scene Simulation, AI Training Data

ACM Reference Format:

Thomas Cansfield, Hannah Simson, Ludwig Gallmeier, Ahmed Ibrahim, Albert Didkovski, Andreas Kolbinger, Anja Schlaak, Benjamin Kass, Daniel Shaquille, Hamza Dursun, Kevin Kuhn, Martin Lauff, Selin Durmus, Surkhay Khanmammadli, Syed Abidi, Tobias Kerner, Vanessa Rieger and Prof. Dr. Torsten Schön. 2025. Weird

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Project Weird Stuff in Traffic, Mar 17 - Jun 24, 2025, Ingolstadt, DE
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2025/06
<https://doi.org/XXXXXXX.XXXXXXX>

Stuff In Traffic. In *Proceedings of Project Weird Stuff in Traffic*. ACM, New York, NY, USA, 21 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Modern object detection models for autonomous vehicles are highly performant in recognizing conventional traffic scenarios. However, real-world environments are inherently unpredictable, and anomalies — such as ducks crossing a motorway or carnival floats with oversized papier-mâché heads — present significant challenges for current object detection systems. Capturing such rare and absurd situations in actual traffic data is statistically unlikely and ethically questionable.[7] Yet, training models to recognize these "weird cases" is crucial to increasing the robustness and safety of autonomous driving systems.

To address this issue, this interdisciplinary project explores an interactive and gamified approach to generating synthetic traffic anomalies and using them to train object detection AI. Users can actively engage in generating absurd traffic scenes using a system based on text-to-image generation models. The system evaluates the generated content by testing whether an object detection model — retrained iteratively with this data — can correctly identify the scene.

This project, conducted as part of a university collaboration, combines expertise in artificial intelligence and user experience design to develop an interactive system for anomaly detection in traffic scenarios. Participants worked jointly on configuring and evaluating generative models capable of producing unusual yet realistic traffic images, as well as on improving object detection systems to recognize such anomalies. In parallel, a gamified application was designed to allow users to input imaginative prompts and track their individual contributions, thereby creating a playful but purpose-driven feedback loop to support model training.

By turning anomaly training into a competition between users and the model, the application not only generates valuable training data, but also fosters user engagement and awareness of the challenges in autonomous driving. This paper presents the system design, technical implementation, user interface, and initial evaluation of the prototype's impact on model robustness and user motivation.

2 Related Work and Motivation

The gamification concept behind "Weird Stuff in Traffic" draws inspiration from prior work in interactive AI systems and creative human-in-the-loop data collection tools. Applications like "Semantris"[1] and "Quick,Draw!"[10] demonstrate how playful interfaces can effectively crowd-source training data for language and vision models. Similarly, platforms like "Artbreeder"[3] show how user-driven image manipulation can produce highly diverse, synthetic visual content, highlighting the creative potential of human-AI collaboration in generative image context. However, few of

these systems target the domain of autonomous driving or focus specifically on anomaly detection in traffic scenes. The project builds on those principles while introducing a novel application area, where playful prompt creation supports the training of AI models for road safety.

3 Gamification Concept and UX Design

The user interface developed for "Weird Stuff in Traffic" transforms anomaly detection into an interactive game. The goal is to allow players to generate unusual traffic scenes that challenge AI models while maintaining a seamless, playful experience. The resulting system combines creativity, instant feedback, and visual clarity to guide users through meaningful interactions with both generative and detection AI components.

3.1 User Flow and Game Logic

The game operates in a fast-paced interaction loop designed for fluid repetition. Users begin by entering a prompt. Either freely or using one of several inspiration tools that support ideation through randomness or narrative guidance. This prompt is then sent to an inpainting model, which returns a selection of images featuring "weird" traffic scenarios. Users then select an image and submit it to the detection model. Afterwards, a score is calculated based on the AI's success or failure, displayed alongside a short avatar feedback and an updated progress bar. Each round works independently. This structure supports both casual play and high-frequency contribution. At each step, from prompt creation to scoring, the system provides immediate and understandable feedback. This highlights how users' creative inputs influence the AI's ability to detect "weird stuff", making their contribution feel both meaningful and impactful.

3.2 Gamification Elements

The platform integrates several game mechanics to maintain engagement and create a meaningful link between gameplay and AI model performance. The main motivation is realized through a point system: Players receive points when the AI fails to detect the anomaly in their selected image. These points are shown immediately after each round. An animated avatar provides emotional feedback and reinforces the interactive character of the experience. It reacts contextually to the AI's performance, using both facial expressions and short text-based comments to create a sense of presence. Through the facial expression, emotions such as surprise, confidence or confusion are shown. This adds an additional layer of emotional response and reinforces the game's interactive character. Early explorations included dynamic avatar progression which were deliberately excluded to avoid misinterpretation about the avatars reflected progression. As a result, the avatar design was kept consistent, while its responses adapt contextually to the AI's performance. The prompt creation is

supported by two distinct input options, each designed to encourage creativity and help produce diverse scenarios. The "Slot Machine" randomly combines descriptive adjectives and objects, offering unpredictable prompts that encourage lateral thinking and highlight edge-case scenarios. The "Flying Words" option allows users to select floating keywords in real time, stimulating spontaneous scene construction. These two options were added to reduce creative fatigue and encourage unexpected prompt combinations, which help for surfacing AI blind spots.

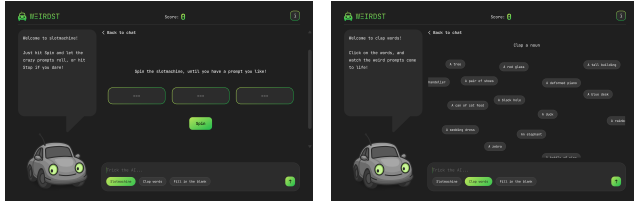


Figure 2. Prompt generation modes: Slot Machine (left), Flying Words (right). Each mode provides a different creative interaction path for generating unusual traffic scenarios.

To further reinforce the sense of contribution, the platform allows users to share their scores, anytime accompanied by the image that led to the current or past scoring, on social media. This feature was introduced to amplify user satisfaction and extend engagement beyond the application context.

3.3 Scoring and Progressing Logic

This scoring logic was developed to provide immediate, transparent feedback while supporting the training goals of the detection AI. Players are rewarded with points when the AI fails to accurately detect the anomaly in the image they selected. This implementation uses a graded scoring model based on the AI's similarity score, a value provided by the detection backend that reflects how confidently AI has identified the anomaly. The fewer elements the AI detects correctly, the higher the score awarded to the player. For example, if the similarity score indicates 60% of the anomaly was recognized, a moderate number of points is granted (e.g. 4 points). If only 20% was detected, a higher reward (e.g. 8 points) is given.

This proportional scoring system encourages players to create increasingly subtle or unconventional prompts, which are more likely to expose limitations in the AI model. After each round the score is displayed and a contextual comment from the avatar adds a narrative. By presenting the score clearly and immediately after each round the system ensures that players understand the consequence of their creative input. This not only supports sustained engagement but motivates them to get better with each round.



Figure 3. Scoring interface: The AI detected the anomaly with 70% confidence, resulting in 3 points. The avatar provides contextual feedback based on the outcome.

3.4 Interface Design

The interface of "Weird Stuff in Traffic" is designed to support fast, focused interaction while reinforcing the system's technical character. The layout follows a structure, top-down flow that leads users intuitively from prompt creation to image selection, detection and feedback, minimizing unnecessary navigation and maximizing engagement with the AI systems. To support creative input, the interface includes two distinct prompt modes: "Slot Machine" and "Flying Words". Each mode presents a simplified, visual rhythm and structure but all lead seamlessly into the same image selection and feedback flow. The image selection view presents AI-generated scenes in a clear grid format, optimized for fast comparison and selection. Once an image is chosen, the detection system analyzes the scene, and the result screen displays a score and an avatar reaction based on the result. A consistent dark green color scheme was chosen for the interface to reflect a clean technical aesthetic aligned with the AI and data-driven nature of the system. JetBrains Mono was selected as the primary typeface, reinforcing the coding-inspired look and feel of the platform. High-contrast text and a restrained use of animation create a focused and functional visual identity. UI elements are arranged to maintain clarity across screen sizes with emphasis on readability and fast input handling. The interface includes an avatar, persistently positioned within the player's view. While its base appearance remains consistent to ensure clarity, it displays animated reactions that correspond to the player's outcomes, such as raised eyebrows, smiles or surprised expressions. Combined with its contextual dialogue, these visual responses add personality without distracting from core interaction. Overall, the interface delivers a lean, functional design that enhances the main interaction loop and supports the platform's underlying purpose: generating and evaluating data for real-world AI improvement through intuitive, user-driven input.

4 Generation

4.1 Data Acquisition

To support the segmentation and inpainting pipeline, a detailed data preparation framework was developed based on the nuScenes dataset. This dataset contains multimodal sensor data, including RGB images, Lidar point clouds, and semantic annotations across 1000 driving scenes, which were filtered based on the previously established operational design domain (ODD). This ODD was defined as following: 'day pictures, good weather, visible road'. The official nuScenes-devkit was utilized with NumPy, OpenCV and PyTorch for extraction, transformation and labeling of the input data for model training and inference. Drivable_area polygons were retrieved from the nuScenes map API in global coordinates. These polygons were initially transformed into the ego perspective frame of the vehicle, using each sample's pose, and later into the corresponding camera frame using the calibrated extrinsics. The transformed polygons were then projected into the image plane through the use of the camera's intrinsic matrix. This enabled the generation of dense and precise drivable_area masks on top of the RGB camera view. OpenCV was used for rendering these masks, which were then used for visualization and supervised training. Lidar point clouds were converted from sensor space into the global frame using the full transformation chain provided in the nuScenes metadata. These points were then rasterized into a fixed-resolution Birds Eye View (BEV) map by quantizing X-Y-coordinates into grid cells. The classification of cells as drivable or non-drivable was determined by their intersection with the projected drivable_area polygons. The BEV maps were used as training input and as a reference for the generation of bounding boxes.



Figure 4. Projection of lidar points from the drivable area onto the corresponding image [6]

For the purpose of object segmentation, a YOLOv11-based pipeline was constructed using processed images and drivable_area masks. For each image, an annotation file containing the normalized coordinates of the target street segments was created. The segmentation model was trained to predict the drivable surface, enabling the selection of bounding

boxes for subsequent inpainting in street view pictures without manual annotations. The sampled inpainting regions were located entirely within the segmented drivable area, with the objective of avoiding occlusions or violations of scene structure. The preprocessing steps, comprising polygon projection, Lidar rasterization, as well as YOLO inference, and file generation, were automated via a Python toolchain. Based on these functions, an inpainting pipeline was developed, enabling the prompt-guided insertion of objects into a street view scene in a creative and realistic way.

4.2 Street detection with YOLO

Given that the selected method involves the generation of objects through inpainting, it is imperative to ensure that the inpainted object is positioned in an appropriate location relevant to road traffic. Therefore, it was necessary to implement a segmentation model that highlights the street in every picture. The segmented area of the street is subsequently used as a reference for the inpainting bounding boxes.

4.2.1 Preparation. The YOLOv11s-seg model was utilized as the base model. The selection of the small YOLO model was predicated on its status as a lightweight model that had been pretrained for segmentation. This characteristic precisely aligned with the requirements of the specified use case, namely the segmentation of the drivable area. Secondly, the process of fine-tuning was more straightforward while maintaining optimal results which made it a suitable choice for that specific application.

The YOLO model was supplied with the previously acquired and filtered images, along with their corresponding annotation files, which amounted to over 400 images. To elaborate, each image, such as "image1.png", was paired with its related annotation file, for example, "image1.txt". These annotation files delineate the road segments within each image.

4.2.2 Ultralytics YOLO. For training, the YOLO trainer python class from the ultralytics library was used. The core of the training setup involved configuring the trainer to understand the dataset's structure and labeling. A configuration file in YAML format specified the dataset location and its format, as well as the only given label, "drivable area". Since the objective was solely to identify road segments, there was only a single class to categorize. This simple configuration allowed the YOLO trainer to correctly load and interpret the dataset.

4.2.3 Data Augmentation. In order to circumvent the pitfalls of overfitting and enhance the model's generalization capabilities, different data augmentation techniques were used. These augmentations involved introducing random variations into the training images, effectively creating new,

unique samples from existing ones. The specific augmentation parameters were integrated directly into the trainer's configuration. Key augmentation strategies included:

- Hue, Saturation and Brightness: Altering these visual properties of the images to expose the model to diverse lighting conditions.
- Scaling: Resizing images to different scales, helping the model learn features at various resolutions.
- Mosaic: A technique in which four images are combined into a single training image, fostering better object detection in crowded scenes and improving contextual understanding. (used on 20% of training pics)
- Shear and Perspective: Distorting images to simulate different viewing angles and geometric variations, enhancing the model's ability to recognize roads under various perspectives.
- Copy-Paste: A technique that involves copying a segment from one image and pasting it into another. (used on 20% of training pics)
- Translation: Shifting the image helps the model become invariant to the precise location of objects within the frame.

While the dataset was initially duplicated multiple times by a factor of 16 to increase the number of training samples, the augmentation process ensured that even these duplicates appeared as unique inputs to the model. Each replicated image was subjected to different augmentation transformations, creating new training examples without the need for manual creation of diverse datasets. This approach was more straightforward to implement with the YOLO trainer than modifying its internal data loading and augmentation pipeline to augment each picture sixteen times. This duplication was only done to use the 24GB of vram from the RTX 4090 more efficiently and reduce the bottleneck caused by the processing steps that the YOLO trainer performed after every epoch.

4.2.4 Output. To assess the effectiveness of the YOLO model, it was tested on a separate set of NuScenes, totalling 1,119 images. Two standard procedures were used for this: Intersection over Union and the Dice Score.

Intersection over Union measures the overlap between the predicted and ground truth regions. It is defined as the ratio between the area of overlap and the area of union of the two masks.

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

Where:

- A is the set of pixels in the ground truth mask (actual street pixels),
- B is the set of pixels in the predicted mask (predicted street pixels),
- $|A \cap B|$ is the number of correctly predicted street pixels (true positives),

- $|A \cup B|$ is the total number of pixels that are either in the ground truth or the prediction (true positives + false positives + false negatives).

The IoU score ranges from 0 to 1:

- IoU = 1 means perfect overlap,
- IoU = 0 means no overlap.

The Dice Score, also known as the F1 score for segmentation, is another measure of overlap, but it places more emphasis on the correctly predicted pixels. It is defined as:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} \quad (2)$$

Where:

- $|A|$ is the number of pixels in the ground truth mask,
- $|B|$ is the number of pixels in the predicted mask,
- $|A \cap B|$ is the number of overlapping pixels between ground truth and prediction.

Like IoU, the Dice score ranges from 0 to 1, where:

- Dice = 1 means perfect prediction,
- Dice = 0 means no overlap.

[8]

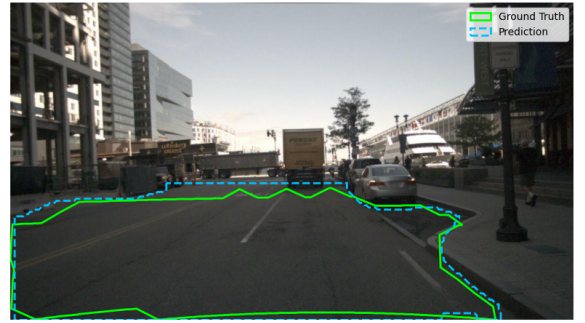


Figure 5. Example picture of YOLO street detection validation (Ground Truth vs. Prediction)

After testing, average Dice and IoU scores of 0.8583 and 0.7539 were obtained, respectively. While these scores could be considered imperfect, this can be attributed to known limitations in the ground truth. When the predicted polygons are compared to the ground truth, it becomes clear that the YOLO model actually outperforms the ground truth. (Figure 5). This is because the ground truth is not 100% accurate. It is calculated based on the lidar points, but lidar points are not available everywhere.

Considering these circumstances, the results demonstrate sufficient accuracy for the intended application.

4.3 Inpainting

Two approaches were considered for the generation process: an inpainting model and a full generation model. The inpainting model was the better choice, because generating

the entire image with a weird prompt might not reflect a real traffic situation. With the full generation model, the entire generated image would be weird, not just the objects themselves.

4.3.1 Inpaint Bounding Box. The first step is to identify the largest unoccupied area on the street. It is important to note that this is necessary to avoid blocking objects, such as automobiles, within the inpaint bounding box. Using the street polygon as the inpaint mask is suboptimal, as it does not allow for the inpainting of objects directly on the street. To illustrate, the generation of an entity such as "a man standing on the street" would be impossible if the street polygon were the sole mask employed.

To address this issue, a method is employed that involves the computation of the largest bounding box that fits entirely within the street polygon. This is achieved through the implementation of a histogram-based approach. This provides the area where the majority of the empty street is situated. In the following step, a random point within the specified bounding box is selected to establish the central point of the inpaint box.

However, a significant issue must be addressed: if the sampled point is located in close proximity to the left or right edge of the bounding box, it may encompass substantial areas outside the polygon, where objects such as automobiles are likely to be found.

In order to mitigate this issue, it is necessary to reduce the width of the largest bounding box by 20%. This adjustment ensures that the random point is sampled more towards the center of the bounding box.

The next problem is that, in most cases, the largest bounding box within the polygon is located at the bottom of the image. This is due to the trapezoidal shape of the streets, which is a result of perspective. However, the objective is not only to inpaint at the bottom of the image; rather, it is to incorporate vacant streets higher up in the image as well. To address this challenge, a modification to the height of the sampled point is required, with adjustments based on the distance between the highest point of the street polygon and the highest point of the largest bounding box. Finally, the dimensions of the inpaint box are established in relation to the dimensions of the background image. This method enables the generation of consistent inpaint boxes that encompass the most empty street area while preserving diversity across different images, as each inpaint box is slightly different.

4.3.2 Inpainting Process. The first inpainting model that was evaluated was Flux1.fill. It offered good general inpainting quality and was easy to use and integrate. However, further testing revealed a consistent issue: the model always tries to fill the entire inpaint area, even when parameters are carefully tuned. For example, if the prompt is "a small pigeon" but the bounding box is large, the result will often contain an oversized pigeon that looks out of place.



Figure 6. Visualizing the inpainting process: The yellow polygon highlights the detected street, with the green bbox showing the largest fit. Light blue indicates the initial sampled point, dark blue its height-adjusted position, and the red bbox is the final inpaint.

One potential workaround is to use a language model to estimate the bounding box size based on the user prompt. This approach, however, introduces several new challenges:

- How does this affect overall performance?
- How reliable is the size estimation for unusual or weird objects?
- What if the user wants to inpaint multiple objects?
- How should multiple bounding boxes be placed?
- What if the objects should interact with each other?

To circumvent these issues, the model was replaced with the RealVisXL5.0 model. It also offers a high inpainting quality, is even faster than Flux1.fill, and most importantly, does not require accurately sized bboxes.

Initial experiments showed that RealVisXL5.0 is capable of inpainting unusual or unseen content, but only with the right parameters. Creative or complex prompts often require different parameter combinations than simple prompts. Using "creative" parameters on an easy prompt can result in either empty outputs or overly abstract results that do not meet expectations.

To address this, different parameter combinations are used for the four different images created per user prompt. This allows to cover a wide range of prompt complexities. The only issue now is that there will probably be images where nothing is inpainted. In that case, the user will have to pick the "best" picture.

Two parameters were varied: *Strength* and *Classifier-Free Guidance Scale (CFG-Scale)*:

- **Strength:** Determines how much noise is added to the input image before the inpainting process begins. Essentially, it controls how much the original image is transformed during the generation. A higher score leads to more change of the base image.
- **CFG-Scale:** Controls how strongly the generated image adheres to the provided text prompt. It's a balance

between following the prompt precisely and allowing the model artistic freedom.

To verify the hypothesis that at least one image per prompt would produce a useful result, a large-scale test was conducted using over 350 weird prompts, resulting in more than 1,400 generated images.

The results confirmed the hypothesis: on average, at least one image per prompt contained a usable inpainting. Two major factors were identified:

1. **Scene Occlusion:** If large parts of the street/inpaint bbox are blocked by big objects (like trucks or other cars), the model struggles to place additional large elements.
2. **Prompt Clarity:** If the prompt is too abstract or ambiguous, the model may misinterpret it or produce irrelevant results.

Naturally, there are limitations to what can be realistically inpainted. Prompts that describe impossible or non-visualizable concepts (e.g., “traffic lights blinking in Morse code”) cannot be generated. Similarly, strongly scene-changing prompts like “a roundabout” or “an intersection made of lava” cannot be fulfilled since the model only inpaints within the specified bbox, not the entire image. Another minor issue is the occasional visibility of the inpaint box edges. This, unfortunately, is also not fixable, especially if the strength parameter is set high.

Overall, the combination of RealVisXL5.0 with multiple parameter combinations yields strong and flexible results. With the figures 7 and 8, some of the best inpainting examples are displayed.



Figure 7. Prompt: A marshmallow man on the street

5 Detection

The core of the anomaly detection system relies on robust object detection models capable of identifying “weird” elements within traffic scenes. This section outlines the training dataset synthesis, development, and comparative analysis for two prominent detection model frameworks, Detectron2 and YOLO, which have been adapted and fine-tuned for the desired application.



Figure 8. Prompt: A swarm of geometric birds made out of glass

5.1 Dataset Synthesis

5.1.1 Dataset Filtering for Realistic Traffic Scenarios.

To support early-stage training of object detection models, a traffic background image dataset was compiled using samples from the nuScenes [6] and BDD100K [18] datasets. Selection criteria focused on front-facing road views captured under clear daylight conditions. In the nuScenes subset, only CAM_FRONT and CAM_BACK frames were retained if the yaw deviation from vehicle orientation remained within $\pm 20^\circ$. A geometric alignment filter was further applied via the nuScenes map API to ensure that the camera’s viewing direction was within a 40° angle of the road vector, according to the defined ODD.

5.1.2 Controlled Image Generation with Stable Diffusion.

In preparation for training pipeline testing, an object image set was created using the AUTOMATIC1111 [4] interface (v1.1.1.1) in combination with the RealisticVision [14] model. Scene inpainting was conducted using rectangular center masks applied to real road scenes, enabling the integration of novel objects such as strollers, dogs, or chairs within traffic scenarios. Positive prompts were enriched with contextual descriptors such as “realistic lighting” or “on asphalt surface” to improve visual consistency, while negative prompts were employed to suppress common failure patterns, including distorted limbs or inconsistent textures.

Generation parameters were standardized for reproducibility and images were generated at a resolution of 640×640 pixels to maintain compatibility with YOLO-based object detection pipelines. Key settings included 25 inference steps, a CFG scale of 7.0, and a denoising strength of 0.75, using the Euler a sampler. All configuration parameters were stored in a shared configuration file (gen_config.json).

Despite using generously sized inpainting masks, the synthetic anomalies were frequently rendered with unexpectedly small dimensions. This often led to misaligned bounding boxes when using automated label generation. To correct this, manual annotations were applied to refine the training labels and ensure accurate alignment with embedded objects.

5.1.3 Dataset Synthesis Outcome. The filtered and generated dataset provided a reliable input for early testing of the "weird" object detection pipelines. While smaller in scale and diversity than the full image generation system, this synthetic set enabled the detection models to be evaluated under realistic and controlled traffic scenarios. The process informed key design and data handling decisions for later training stages.

5.2 Ultralytics YOLO

A careful selection from the rapidly evolving landscape of YOLO architectures was needed. Ultralytics[15] continually releases new YOLO versions (e.g., v8, v9, v10, 11, 12), each with distinct architectural designs. A key challenge is that newer versions do not inherently guarantee superior performance across all datasets; optimal performance is often dataset-dependent. Therefore, a systematic approach was crucial to identify the most suitable YOLO variant for the synthesized dataset.

5.2.1 YOLO Model Selection. A systematic evaluation methodology was developed that can be applied to current and future Ultralytics YOLO releases with different model sizes (e.g. nano, small, medium, etc.) to ensure the most fitting model for a given dataset. A approach consisted the following steps:

1. **Initial Base Model Selection:** Evaluation began with the selection of a base model, YOLO11n, chosen for its balance of performance and computational efficiency.
2. **Full Dataset Training and Evaluation:** The chosen base model was trained and evaluated on the entire dataset.
3. **Performance Monitoring:** Its performance was monitored on a dedicated test set, specifically focusing on the mAP50 metric.
4. **Decision Point:**
 - If the performance was deemed adequate (e.g., mAP50 > 0.96), this model would be selected for deployment. This acts more as a heuristic, but can save significant effort in testing other models.
 - If the performance was not adequate, it indicated a need to explore other models.
5. **Cross-Validation with Dataset Splits:** To ensure a comprehensive comparison, N different splits of the dataset was created. This effectively served as a cross-validation strategy, exposing each model to varied subsets of the data.
6. **Comparative Training and Evaluation:** All relevant YOLO versions of the same size were trained on each of these dataset splits. Crucially, identical training and augmentation parameters were employed across all runs to ensure fairness and prevent bias in the comparison.

7. Results Aggregation: The performance metrics from all runs and models were aggregated as the mean of N runs.

The aggregated results for key metrics are presented in Table 1:

Metric (mean)	YOLOv8n	YOLOv9t	YOLOv10n	YOLO11n	YOLO12n
mAP50-95	0.630	0.641	0.620	0.635	0.633
mAP50	0.934	0.940	0.925	0.942	0.936
Recall	0.904	0.914	0.897	0.914	0.907
Precision	0.938	0.943	0.928	0.939	0.937

Table 1. Comparative Performance of Different YOLO Models Across 5 Splits

Although YOLOv9t showed slightly better performance in some metrics compared to YOLO11n, a critical factor in model selection for deployment was training efficiency. YOLOv9t required significantly more training time per epoch (25.2s vs. 12.81s for YOLO11n) due to its substantially larger number of layers (544 vs. 181). Attempts to further fine-tune YOLOv9t did not yield impressive performance gains and struggled to converge within reasonable timeframes (200-300 tuning runs).

Ultimately, YOLO11n was selected as proposed candidate model for "weird" object detectoin task. It demonstrated better overall performance with tuned hyperparameters, converging reliably within 200 epochs. Its more efficient training time also made it significantly more suitable for integration into the iterative feedback loop of the gamified application, where rapid retraining and deployment are beneficial for continuous model improvement.

5.2.2 YOLO Hyperparameter Tuning. Effective hyperparameter tuning is critical for achieving optimal performance in deep learning models. For YOLO11n, this presented a significant challenge due to the large number of tunable parameters, including approximately 12 augmentation parameters and 12-15 optimization-related parameters, totaling 25-30 variables to adjust prior to training. While Ultralytics' 'base model.tune()' method offers built-in hyperparameter optimization using Genetic Algorithms (GA), GAs typically require a substantially higher number of runs (e.g., 500-1000) to converge compared to more efficient methods like Bayesian Optimization or Hyperband (e.g., 200-500 runs). Given that each training run could span 50 epochs, relying solely on GA would result in prohibitively long tuning times.

To overcome this limitation, the Optuna[2] library was selected, which defaults to Bayesian Optimization, specifically using the Tree-structured Parzen Estimator (TPE) algorithm. This allowed for more efficient exploration of the hyperparameter space. A custom 'tune.py' script was developed that iteratively trained the YOLO model with sets of hyperparameters proposed by Optuna. The objective function was set to maximize the mAP50-95 score from the test set, as this

metric provides a comprehensive evaluation of both localization and classification accuracy across various Intersection over Union (IoU) thresholds. The study was conducted for a maximum of 300 iterations, with continuous monitoring for convergence.

The optimization history plot (Figure 9) revealed that significant improvements in mAP50-95 plateaued after approximately 205 runs, indicating that further tuning would yield negligible gains.

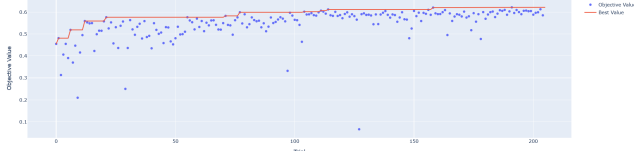


Figure 9. Optimization history plot for YOLO11n hyperparameter tuning using Optuna. The x-axis represents the number of trials, and the y-axis represents the mAP50-95 score.

The best performing trial, run #191, achieved an mAP50-95 of 0.6216. A closer examination of the best 5 trials highlighted the consistent use of the *Adam* optimizer, a batch size of 16, and strong momentum (around 0.9 or higher) contributed to higher performance.

Optuna’s hyperparameter importance analysis revealed key influencers on model performance: optimizer choice, specific augmentation parameters (hsv_h, box, scale, degrees), initial learning rate (lr0), and layer freezing (freeze). Conversely, dropout, mixup, and cutmix showed lower importance.

5.2.3 YOLO Training and Evaluation. After selecting YOLO11n and performing extensive hyperparameter tuning using Optuna, the model was trained for 300 epochs with a patience of 50 epochs (meaning training would stop if performance on the validation set did not improve for 50 consecutive epochs). The tuned hyperparameters from the best Optuna run (#191) were used. The results, when compared to the model trained with default hyperparameters (before tuning), demonstrate a significant improvement in detection performance across various metrics.

The table below summarizes the key performance indicators before and after hyperparameter tuning, along with the percentage gain for each metric:

The post-tuning results show a notable improvement across all core detection metrics. The F1-score, a harmonic mean of precision and recall, increased from 0.88 to 0.91, indicating a better balance between correctly identifying anomalies and minimizing false positives/negatives. This optimal F1-score was achieved at a slightly lower confidence threshold (0.516 vs 0.574), suggesting the tuned model is more confident in its predictions at a broader range.

Metric	Before Tuning	After Tuning	% Gain
F1-Score (at optimal confidence)	0.88	0.91	3.0%
Optimal F1 Confidence	0.574	0.516	–
Precision	0.908	0.948	4.0%
Recall	0.842	0.886	4.4%
mAP50	0.915	0.944	2.9%
mAP50-95	0.596	0.612	1.6%
Normalized Confusion Matrix			
Anomaly (True Positive Rate)	0.90	0.94	4.0%
Background (False Positive Rate)	0.10	0.06	4.0%

Table 2. Comparative Performance of YOLO11n Before and After Hyperparameter Tuning

Precision gains meant fewer irrelevant objects were identified as anomalies. While Recall improvement indicated that the model became better at detecting a higher proportion of actual anomalies present in the images. The mAP50-95, which averages mAP over various IoU thresholds, also saw a modest but important gain of 1.6%, showcasing improved localization accuracy across different levels of overlap with ground truth.

Crucially, the confusion matrix further highlights the benefits of tuning. The true positive rate for detecting the anomaly class increased from 0.90 to 0.94, meaning the model correctly identified 94% of the actual anomalies. Concurrently, the false positive rate (i.e., predicting anomaly when it was background) decreased from 0.10 to 0.06. This indicates a substantial improvement in the model’s ability to discriminate between true anomalies and normal background elements, leading to fewer incorrect detections and ultimately a more robust anomaly detection system.

These enhancements validate the effectiveness of the selected hyperparameter tuning strategy and confirm that the YOLO11n model is well-optimized for the task of identifying anomalies in traffic scenes.

5.3 Detectron2 Detection Model

5.3.1 Detectron2 Model Selection. The faster_rcnn_X_101_32x8d_FPN_3x configuration from Detectron2[17] was selected as a baseline for the object detection pipeline. This architecture combines a ResNeXt-101–32×8d backbone with a Feature Pyramid Network (FPN) and a Region Proposal Network (RPN), making it effective for multi-scale object detection. Among the available Faster R-CNN models in the Detectron2 model zoo that utilize precomputed RPN proposals, this variant achieved the highest bounding box Average Precision (mAP) of 40.6, outperforming other baseline models such as faster_rcnn_R_50_FPN with mAP: 36.8 and faster_rcnn_R_101_FPN with mAP: 39.0. The performance made it a suitable candidate for detecting the synthetically generated “weird objects”. Therefore, this model baseline was adopted to ensure a high performance for the detection task.

5.3.2 Detectron2 Hyperparameter Tuning. To achieve optimal object detection, hyperparameter tuning is essential as the Bayesian Optimization method from the Optuna library was applied. Tuning was performed on a fixed 640x640 image size, where the learning rate, optimizer, weight decay, learning rate decay factor, learning rate decay steps, region of interest (RoI) sampling, and batch size hyperparameters were explored with a target objective function of maximizing the mean average precision (mAP). During hyperparameter turning, the best trial achieved an mAP of 66.67 with the following hyperparameter values:

- learning rate: 0.0008,
- optimizer: AdamW,
- weight decay: 0.00085,
- learning rate decay factor: 0.1775
- learning rate decay steps at iterations: 2000, 3500, 4000
- RoI Sampling: 96
- Batch Size: 4

5.3.3 Detectron2 Training and Evaluation. After achieving an initial hyperparameter tuning, an initial Detectron2 model was trained for 10 epochs using the optimal hyperparameters. Here, a mAP50 score of 0.777 and a mAP50-95 score of 0.502 was achieved. A further evaluation was conducted between two Detectron2 models, with one using suggested hyperparameters while the other used the optimal hyperparameters. A table showing the comparison of the two is found below.

Metric	Before Tuning	After Tuning	% Gain
mAP50	0.777	0.938	20.71%
mAP50-95	0.502	0.670	33.47%
Normalized Confusion Matrix			
Anomaly (True Positive Rate)	0.96	0.98	4.0%
Background (False Positive Rate)	0.04	0.02	-4.0%

Table 3. Comparative Performance of Detectron2 Before and After Hyperparameter Tuning

5.4 Model Comparison and Selection

The selection for the single, implemented detection model was determined by the model's mAP performance, inference times, and ease of implementation. Ultimately, Detectron2 was selected due to its impressive mAP performance, simple implementation, and reasonable inference times.

After selecting Detectron2 as the implemented model, a hyperparameter tuning using the Optuna library was conducted with a dataset consisting of 2,974 training samples, 372 validation samples, and 372 test samples. A total of 200 trials were performed, each running for 6,000 iterations where every 500 iterations, validation was performed with the objective of maximizing the mAP. To enable improved performance, image sizes of 1600x800 were used, and an additional

Metric	Detectron2	YOLO11n
Prediction Accuracy		
mAP50	0.938	0.944
mAP50-95	0.670	0.612
Normalized Confusion Matrix		
Anomaly (True Positive Rate)	0.98	0.94
Background (False Positive Rate)	0.02	0.06
Model Efficiency & Resources		
Parameters (M)	104.8	2.6
Avg. inference time/image* (ms)	40.0	3.0

*Tested on a CUDA H100 GPU.

Table 4. Comparative Performance of Detectron2 vs. YOLO11n

hyperparameter was considered: Non-Maximum Suppression (NMS). Across all trials, the highest mAP value was 52.50 with the following hyperparameter values:

- learning rate: 0.0009995,
- optimizer: SGD w/ momentum of 0.9466,
- weight decay: 0.0001,
- learning rate decay factor: 0.22561
- learning rate decay steps at iterations: 2000, 3500, 4000
- RoI Sampling: 96
- Batch Size: 2

After the hyperparameter tuning, a final training was performed consisting of a warm-up phase of 1,000 iterations and 15,000 normal iterations using the found optimal hyperparameters. By applying image augmentation, configuring the anchor generator with varying scales and aspect ratios, and setting the RPN feature map to 5 different levels of the FPN, objects of various sizes and shapes were effectively detected.

A final evaluation was conducted using a confusion matrix on the test dataset, and the inference was run with varying confidence thresholds to identify the optimal threshold that minimized both false positives and false negatives. The final model achieved an average inference time of 40 ms, with an mAP of 51.904, making it suitable for near real-time detection of unusual objects in street scenes.

6 Implementation

To facilitate the user experience and gamification concepts, an engaging, user-centered frontend featuring a chat interface was developed to support the core goal of improving object detection models through interactive user input and feedback. The design and implementation choices were informed by modern web development practices, emphasizing performance, maintainability, and scalability. A backend was developed to accommodate the functionality of image generation and "weird" object detection and to provide the user

with a score based on the ability to correctly detect the user's desired image prompt within the generated image.

6.1 Frontend

6.1.1 Technology Stack and Architecture. The application was developed using the Next.js framework [16], which extends React[9] with features such as server-side rendering and built-in API routing. This enabled fast load times and seamless integration between frontend and backend components. React's component-based architecture enabled modular development and the creation of reusable, consistent UI components.

TypeScript was used throughout the project to enforce static typing, reduce runtime errors, and improve code robustness. For styling, Tailwind CSS[11] was adopted to implement a utility-first design approach, which accelerated UI development and ensured consistent styling.

6.1.2 API Integration. The frontend communicated with backend services via RESTful API endpoints implemented using Next.js API routes. Two primary endpoints were utilized: `/api/generate` for synthetic image generation and `/api/detect` for anomaly detection based on user input.

The `/api/generate` route accepted a user-provided textual prompt and returned an array of Base64 encoded images:

```

1 // Request body
2 const requestBody = {
3   prompt: "textual prompt provided by the user"
4 };
5
6 // Type for a single generated image
7 export interface GeneratedImage {
8   prompt: string;
9   imageBase64: string;
10 }
11
12 // Response containing multiple generated images
13 export interface GeneratedImages {
14   images: GeneratedImage[];
15 }

```

After image selection, the `/api/detect` endpoint was called with both the original prompt and the selected image. The response included a similarity score and a processed image, also Base64 encoded:

```

1 // Request body
2 const requestBody = {
3   prompt: "Associated textual prompt",
4   imageBase64: "... //selectedImage"
5 };
6
7 // Response type
8 type DetectApiResponse = {

```

```

9   prompt: string;
10  score: number;
11  imageBase64: string; //detectedImage
12 };

```

6.1.3 State Management and UI Logic. To orchestrate frontend logic, API communication, and dynamic UI behavior, a global state management system using "Zustand"[12] was implemented. "Zustand" provides a lightweight, boilerplate-free state management solution that replaces the need for complex React Context providers and reduces prop drilling. The state is organized into specialized stores such as `gameStore.ts`, `uiStore.ts`, and game-specific stores for different mini-games.

The main `gameStore` manages core application states including:

- `messages` – storing user inputs and AI responses,
- `isLoadingGeneration` – tracking pending operations,
- `earnedPoints` - accumulating scores when the AI model fails to correctly detect or interpret user-generated images,
- `carAnimationState` – controlling avatar animations based on detection results.

The "Zustand" stores encapsulate both state and actions, including complex asynchronous operations like `handleGenerate` and `handleImageSelect`. Utility functions compute rewards and adjust training progress based on similarity scores from the detection API. This architecture ensures real-time feedback, maintains the gamified nature of the platform, and provides persistent state management through session storage integration.

6.2 Backend

6.2.1 Technology Stack and Architecture. To reduce the complexity of integration, a Python-based backend was developed using the FastAPI framework where necessary models were initialized on startup, and services, such as image generation and image detection, can be asynchronously called from the frontend via RESTful APIs. The backend comprises five distinct components: a subject extractor, street detector, image Generator, object detector, and object summarizer. While the YOLO-based Street Detector, RealVisXL5.0 Image Generator, and Detectron2-based object detector have been previously discussed, the subject extractor and object summarizer play a critical role in the scoring process. In order to provide the user with an understandable score, it is imperative that a comparison between the user's image request and a summary of the detected weird object is performed. To accomplish this, the user's prompt is given to a prompt extractor, which isolates relevant terms. The extractor is based on `spaCy`, an optimized natural language processing library. Its small English pipeline, `en_core_web_sm`, handles all necessary operations internally, including tokenization,

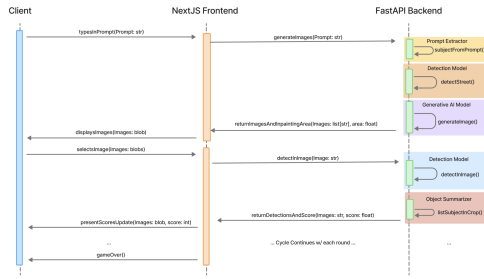


Figure 10. Frontend-Backend Interaction Sequence Diagram

part-of-speech tagging, and lemmatization. Additionally, a hardcoded list of excluded terms was incorporated. These terms mainly describe background elements of traffic scenes, such as "sidewalk" and "horizon," and are irrelevant to the scoring method. After the user selects a generated image, the image is then sent to the backend's object detector where a bounding box for the weird object within the scene is extracted. Using the bounding box coordinates, a copy of the generated image is cropped to exclusively display the weird object, which is then passed, along with a fixed system prompt, to the object summarizer. Under the hood, the object summarizer utilizes the Qwen2-7B-Instruction vision language mode (VLM) to describe the objects present within the cropped image in the form of a string-containing list. This, along with the list generated by the prompt extractor, is used for scoring where the recall is computed as the proportion of relevant object descriptions identified within the prompt extraction output. Scoring is based on the following logic:

```

# Scoring
if boxes:
    if recall != 0.0:
        score = 50.0 + round(50 * recall, 2)
    else:
        score = 50.0
else:
    score = 0.0

```

6.3 Deployment

A shared GIT repository containing the frontend, backend, model weights, and background images necessary for image generation was hosted on a virtual machine provided by the university. Here, it could utilize a 48GB Nvidia H100 to perform image generation and detection at rapid speeds, allowing for a more optimal user experience with lower waiting times. To simplify the use of this project, bash scripts were created to automate environment setup and simultaneously start the frontend and backend for which SSH tunneling could then be used for interaction.

7 User Study / Evaluation

To assess the effectiveness of the interface and interaction model, a structured user study was conducted with 25 participants from diverse academic and technical backgrounds. The goal of the study was to evaluate usability, satisfaction, and clarity of interaction. Special attention was paid to assessing the effectiveness of the UI design itself, including layout and color perception. Participants first completed demographic survey, followed by a guided walkthrough using a clickable Figma prototype. During the session, the Think-Aloud method was used to gather real-time verbal feedback. After the interaction phase, participants completed two standardized questionnaires: the System Usability Scale (SUS) and the User Experience Questionnaire (UEQ).

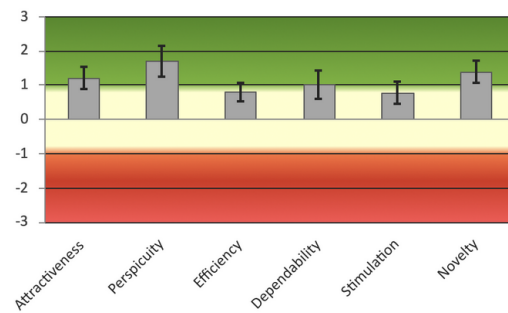


Figure 11. UEQ[13] results across six dimensions. Participants rated the interface particularly high on perspicuity and novelty.

In the initial round of testing, the System Usability Scale (SUS) showed a score of 71.1 indicating above-average usability. After implementing improvements based on participants' feedback — including a simplified avatar dialogue and a new onboarding modal — the final evaluation showed a higher SUS score of 85.5, reflecting excellent usability.

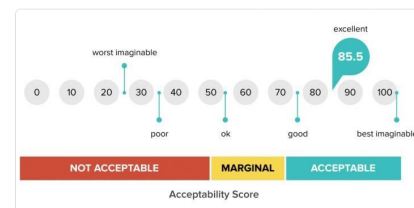


Figure 12. System Usability Scale (SUS)[5] score of 85.5, indicating excellent overall usability based on participant ratings.

Participants responded positively to the game's interface and interaction logic, particularly praising the dark green

theme and the simplicity of the input methods. The “Slot Machine” and “Flying Words” inspirations were highlighted as enjoyable and effective prompt generators. At the same time, feedback indicated that some users found the scoring feedback too abstract. In response, text labels were revised to clarify point distribution and the avatar dialogue was simplified to avoid misunderstandings. To improve understanding of the game’s logic and goals, a modal window was added after the user study. This modal provided a specific introduction to the game’s purpose, a brief explanation of key interface elements and clear instructions on how to play. The goal was to improve orientation for first-time users and ensure a shared understanding of the mechanics. These insights were integrated into the final prototype and retested with a concluding study. The evaluation confirmed both the effectiveness of the improvements and that the main functionality of the application was clearly understood by the participants.

8 Discussion

The evaluation results showed that the interface successfully connects user interaction with the goal of training an AI system. Users were motivated to create unusual prompts. The combination of scoring and direct feedback helped users understand their role in improving AI. A key aspect of the system is the use of creativity to challenge AI. Players are rewarded when they manage to generate scenes that the detection model fails to recognize correctly. This makes the experience playful while highlighting challenges for AI in traffic situations. Users reported that they quickly grasped the idea of “outsmarting the AI” and found it motivating. The interface design supported this well. Despite offering different ways to create prompts, the user experience remained consistent and easy to follow. The animated avatar also contributed to a more engaging experience. By visually reacting to the player’s performance with expressions such as surprise, pride, or confusion, it added a human-like layer of feedback that complemented the scoring and text-based responses. This helped users feel more emotionally connected to the system while staying focused on the core gameplay. Overall, the system achieved its goal of making AI improvement accessible, interactive, and meaningful through a well-designed gaming interface.

9 Limitations and Future Work

While the application fulfilled its main design goals, several limitations became evident during testing and evaluation. One major limitation concerns the transparency of AI feedback. Although the similarity score drives the point calculation, users often did not understand why a certain detection succeeded or failed. Future versions could address this through visual overlays or heatmaps indicating detected areas. A second limitation concerns long-term engagement. In its current form, the application supports session-based

play but lacks features such as progress tracking or social elements like leaderboards. Introducing a lightweight account system or a leaderboard system could help increase user engagement and contribution. Additional technical improvements are planned for the frontend. These include implementing a user authentication system for personalization and progress tracking. Furthermore, integrating a dedicated state management solution, such as a Redux, is being considered to support more complex application logic as the platform evolves. In terms of accessibility additional work is needed to support alternative input methods (e.g. speech or image prompts). While the system is designed to simulate AI learning progression through gamified feedback, it does not implement real-time model updates. The anomaly detection model does not improve continuously during gameplay. Instead, the user-generated images are stored and required manual curation for future retraining phases. This approach, while ensuring higher data quality and control, limits the immediacy of AI adaptation and the potential for direct user impact on model performance. Additionally, evaluating the authenticity and utility of generated images for training remains a subjective and resource-intensive task. Future development could explore integrating a semi-automated retraining pipeline that allows periodic updates to the anomaly detection model using selected user-generated data. Such integration would help close the loop between user interaction and model improvement more effectively. Moreover, implementing feedback mechanisms that inform users about the long-term impact of their contributions could further enhance motivation and trust. Expanding the system to include edgecase detection in other safety-critical domains, such as robotics or industrial automation, could also broaden the applicability and relevance of the concept. Overall, the project demonstrated that gamified AI interaction can be both user-friendly and functionally relevant. Building on this foundation, future iterations could evolve into a scalable platform for real-world AI training through crowd-sourced creativity.

10 Conclusion

This project presents a novel approach to enhancing object detection systems for autonomous driving through the integration of user-generated content and gamified interaction. By allowing users to create unusual traffic scenarios via a prompt-based interface, the system enables the collection of diverse and challenging training data. The gamification elements, such as scoring mechanisms and AI progress indicators, provide motivation while simultaneously educating users about the current limitations of detection models.

The interdisciplinary collaboration between user experience designers and AI developers resulted in a solution that balances technical functionality with user engagement. The system illustrates the potential of human-AI collaboration

in anomaly detection for real-world applications. Future development may focus on scaling data collection, refining evaluation methods, and implementing long-term motivational strategies to support continuous improvement of AI capabilities.

References

- [1] AI, G. [2018], ‘Semantris’, <https://research.google.com/semantris/>. [Accessed 21-06-2025].
- [2] Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. [2019], Optuna: A next-generation hyperparameter optimization framework, in ‘Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)’.
- [3] Artbreeder [n.d.], ‘Create images with ai’, <https://www.artbreeder.com>. [Accessed 20-06-2025].
- [4] AUTOMATIC1111 [2022], ‘Stable diffusion web ui’, <https://github.com/AUTOMATIC1111/stable-diffusion-webui>. Accessed June 2025.
- [5] Brooke, J. [1996], ‘System usability scale (sus)’, <https://www.usability.gov/how-to-and-tools/methods/system-usability-scale.html>. [Accessed 21-06-2025].
- [6] Caesar, H., Bankiti, V., Lang, A. H., Vora, S., Liong, V. E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G. and Beijbom, O. [2019], ‘nusenes: A multi-modal dataset for autonomous driving’, *arXiv preprint arXiv:1903.11027*.
- [7] Fessel, K. [2019], ‘5 significant object detection challenges and solutions’, *Medium, TDS Archive*. [Accessed 23-06-2025].
- [8] Huynh, N. [2023], ‘Understanding evaluation metrics in medical image segmentation’, https://medium.com/@nghihhuynh_37300/understanding-evaluation-metrics-in-medical-image-segmentation-d289a373a3f. [Accessed 22-06-2025].
- [9] Inc., M. P. [2023], ‘React – a javascript library for building user interfaces’, <https://react.dev>. [Accessed 21-06-2025].
- [10] Lab, G. C. [2017], ‘Quick, draw!’, <https://quickdraw.withgoogle.com>. [Accessed 21-06-2025].
- [11] Labs, T. [2023], ‘Tailwind css’, <https://tailwindcss.com>. [Accessed 21-06-2025].
- [12] Poimandres [2023], ‘Zustand documentation - introduction’, <https://zustand.docs.pmnd.rs/getting-started/introduction>. [Accessed 21-06-2025].
- [13] Schrepp, M.; Hinderks, A. T. J. [2014], ‘User experience questionnaire handbook’, <https://www.ueq-online.org>. [Accessed 21-06-2025].
- [14] SG161222 [2023], ‘Realistic vision v5.1 (no vae) - inpainting model’, https://huggingface.co/SG161222/Realistic_Vision_V5.1_noVAE. Accessed June 2025.
- [15] Ultralytics [2024], ‘Ultralytics YOLO Documentation’, <https://docs.ultralytics.com/models/>. Accessed on July 26, 2025.
- [16] Vercel [2023], ‘Next.js documentation’, <https://nextjs.org/docs>. [Accessed 21-06-2025].
- [17] Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y. and Girshick, R. [2019], ‘Detectron2’, <https://github.com/facebookresearch/detectron2>. Accessed on July 26, 2025.
- [18] Yu, F., Chen, H., Wang, X., Xian, W., Chen, Y., Liu, F., Madhavan, V. and Darrell, T. [2020], ‘Bdd100k: A diverse driving dataset for heterogeneous multitask learning’.
URL: <https://arxiv.org/abs/1805.04687>

Appendix

Figma Resources Two collaborative Figma boards were used throughout the project to structure, design, and evaluate the application. Together, they provide a complete picture of

the development process. From early exploration to final UI design and testing.

1. Final Design Board <https://figma.com> – Design Board

This board contains:

- High-fidelity screen flows
- Design system and reusable components
- Color styles, typography and layout structure
- Final UI prototypes

2. FigJam Board <https://figma.com> – FigJam Board

This board contains:

- Project Roadmap
- Brainstorming outputs and user flow drafts
- Early wireframe sketches and explorations
- Prompt mode concept variations
- Annotated user study results (SUS, UEQ)
- Feedback from internal review sessions

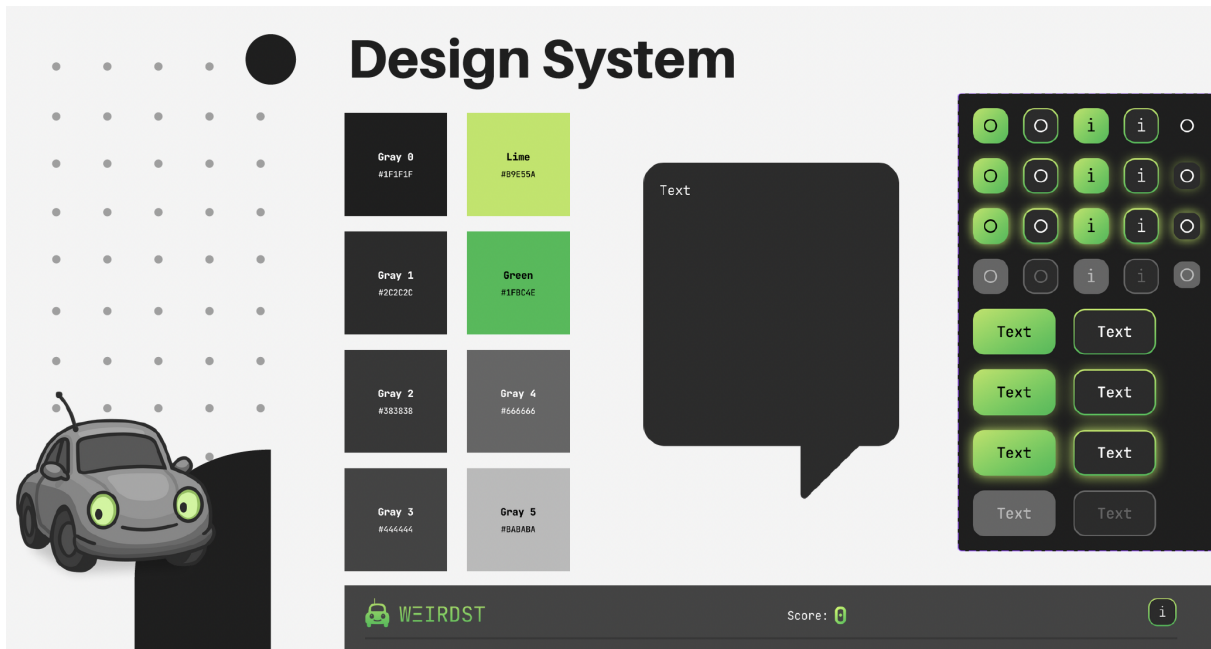


Figure 13. Overview of the design system including UI components, color codes, and visual consistency guidelines.

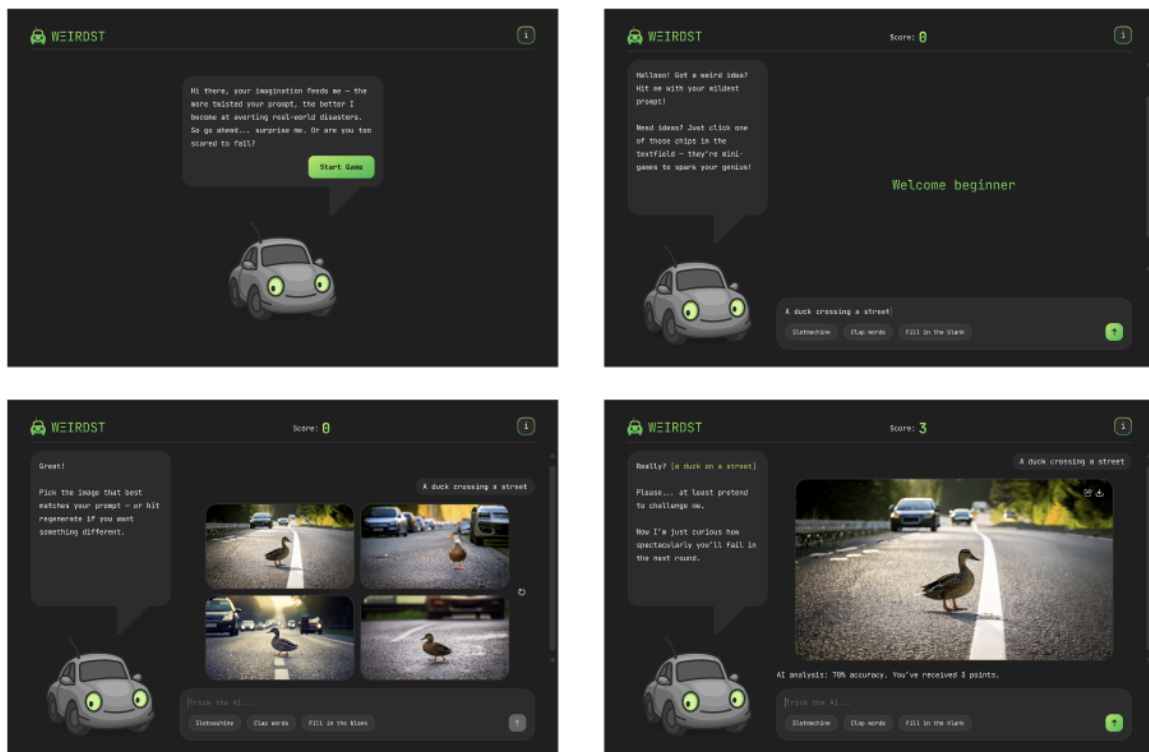


Figure 14. Sequence of game screens: from starting the challenge, prompt entry, image selection, to final AI evaluation and score.

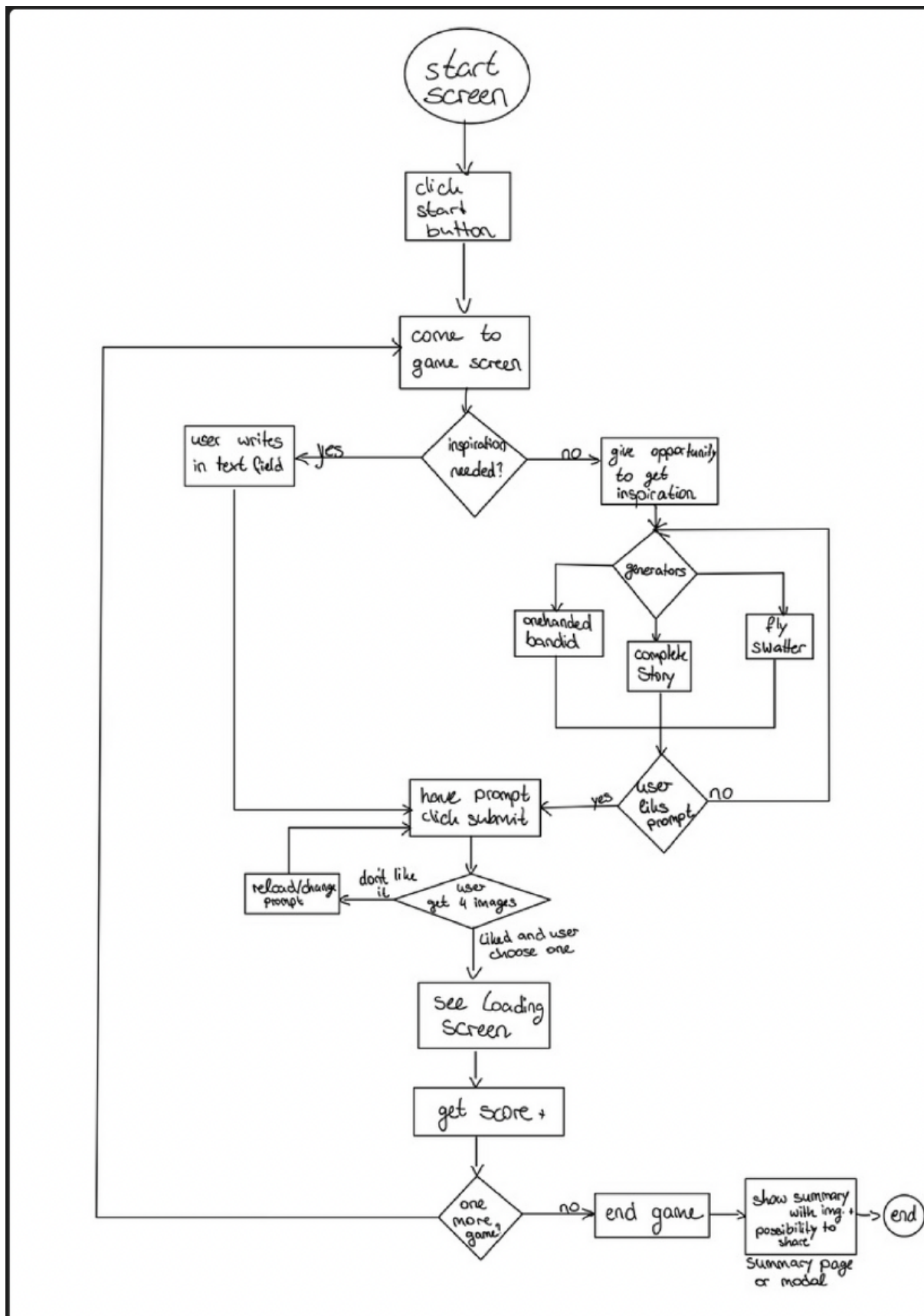


Figure 15. Game flowchart visualizing the user journey from launching the application to scoring and feedback stages.

System Usability Scale Questionnaire	Strongly Disagree				Strongly Agree
1. I think that I would like to use this product frequently.	1	2	3	4	5
2. I found the product unnecessarily complex.	1	2	3	4	5
3. I thought the product was easy to use.	1	2	3	4	5
4. I think that I would need the support of a technical person to be able to use this product.	1	2	3	4	5
5. I found the various functions in the product were well integrated.	1	2	3	4	5
6. I thought there was too much inconsistency in this product.	1	2	3	4	5
7. I imagine that most people would learn to use this product very quickly.	1	2	3	4	5
8. I found the product very awkward to use.	1	2	3	4	5
9. I felt very confident using the product.	1	2	3	4	5
10. I needed to learn a lot of things before I could get going with this product.	1	2	3	4	5

Figure 16. System Usability Scale (SUS) questionnaire presented to users to evaluate perceived usability.

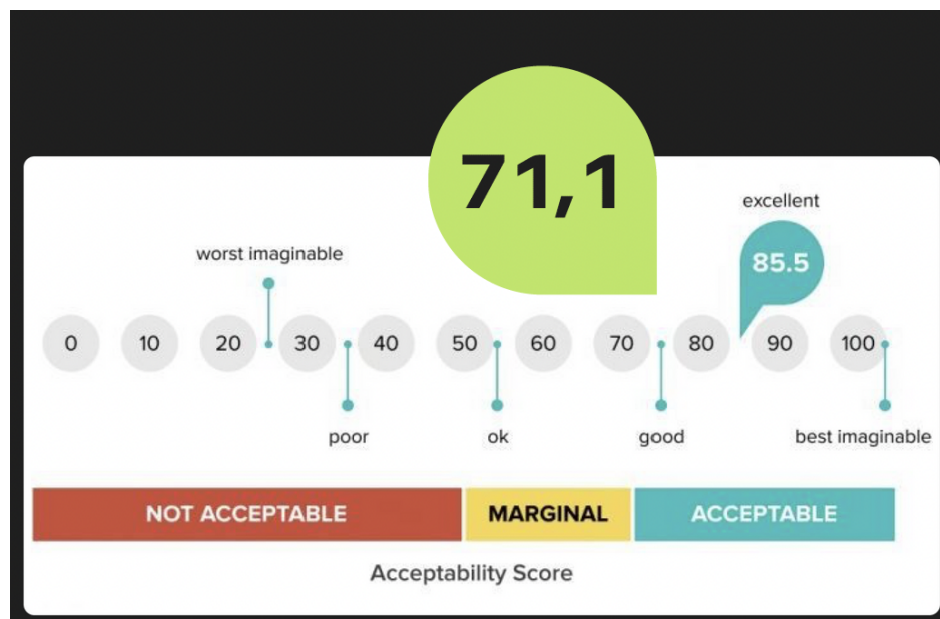


Figure 17. SUS score visualization showing a result of 71.1, categorized as “Good” usability.

	1	2	3	4	5	6	7		
annoying	☐	☐	☐	☐	☐	☐	☐	enjoyable	1
not understandable	☐	☐	☐	☐	☐	☐	☐	understandable	2
creative	☐	☐	☐	☐	☐	☐	☐	dull	3
easy to learn	☐	☐	☐	☐	☐	☐	☐	difficult to learn	4
valuable	☐	☐	☐	☐	☐	☐	☐	inferior	5
boring	☐	☐	☐	☐	☐	☐	☐	exciting	6
not interesting	☐	☐	☐	☐	☐	☐	☐	interesting	7
unpredictable	☐	☐	☐	☐	☐	☐	☐	predictable	8
fast	☐	☐	☐	☐	☐	☐	☐	slow	9
inventive	☐	☐	☐	☐	☐	☐	☐	conventional	10
obstructive	☐	☐	☐	☐	☐	☐	☐	supportive	11
good	☐	☐	☐	☐	☐	☐	☐	bad	12
complicated	☐	☐	☐	☐	☐	☐	☐	easy	13
unlikable	☐	☐	☐	☐	☐	☐	☐	pleasing	14
usual	☐	☐	☐	☐	☐	☐	☐	leading edge	15
unpleasant	☐	☐	☐	☐	☐	☐	☐	pleasant	16
secure	☐	☐	☐	☐	☐	☐	☐	not secure	17
motivating	☐	☐	☐	☐	☐	☐	☐	demotivating	18
meets expectations	☐	☐	☐	☐	☐	☐	☐	does not meet expectations	19
inefficient	☐	☐	☐	☐	☐	☐	☐	efficient	20
clear	☐	☐	☐	☐	☐	☐	☐	confusing	21
impractical	☐	☐	☐	☐	☐	☐	☐	practical	22
organized	☐	☐	☐	☐	☐	☐	☐	cluttered	23
attractive	☐	☐	☐	☐	☐	☐	☐	unattractive	24
friendly	☐	☐	☐	☐	☐	☐	☐	unfriendly	25
conservative	☐	☐	☐	☐	☐	☐	☐	innovative	26

Figure 18. User Experience Questionnaire (UEQ) form used to assess multiple UX dimensions via semantic differential scales.

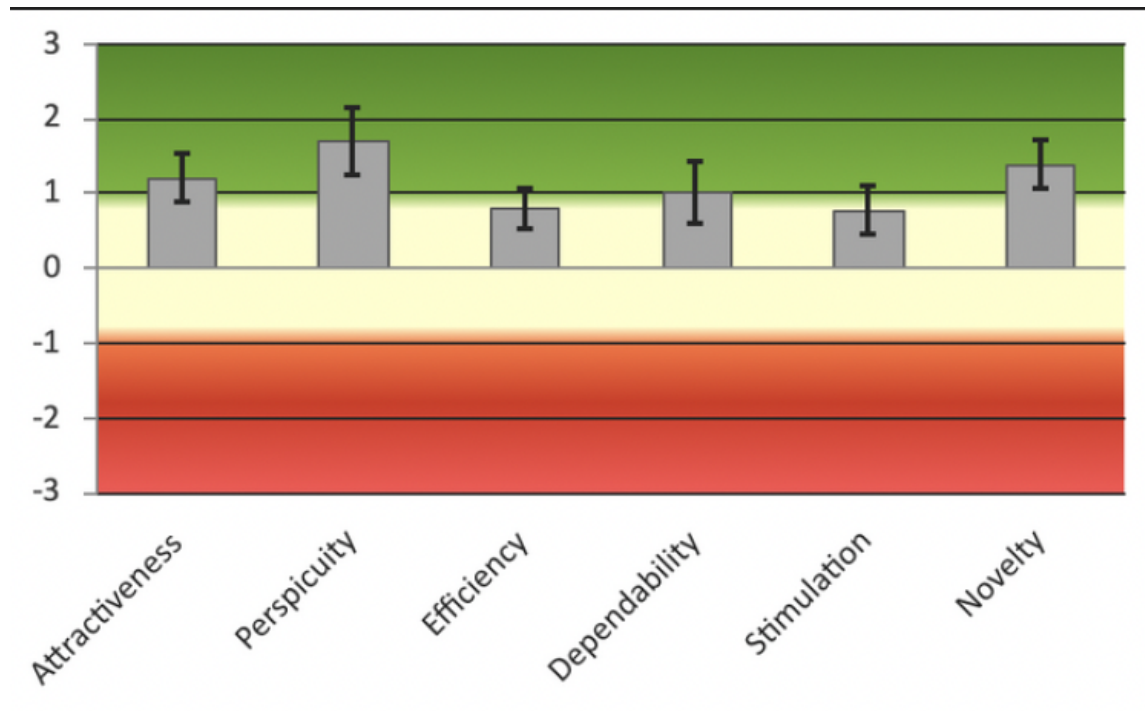


Figure 19. UEQ results showing user ratings across Attractiveness, Perspicuity, Efficiency, Dependability, Stimulation, and Novelty.

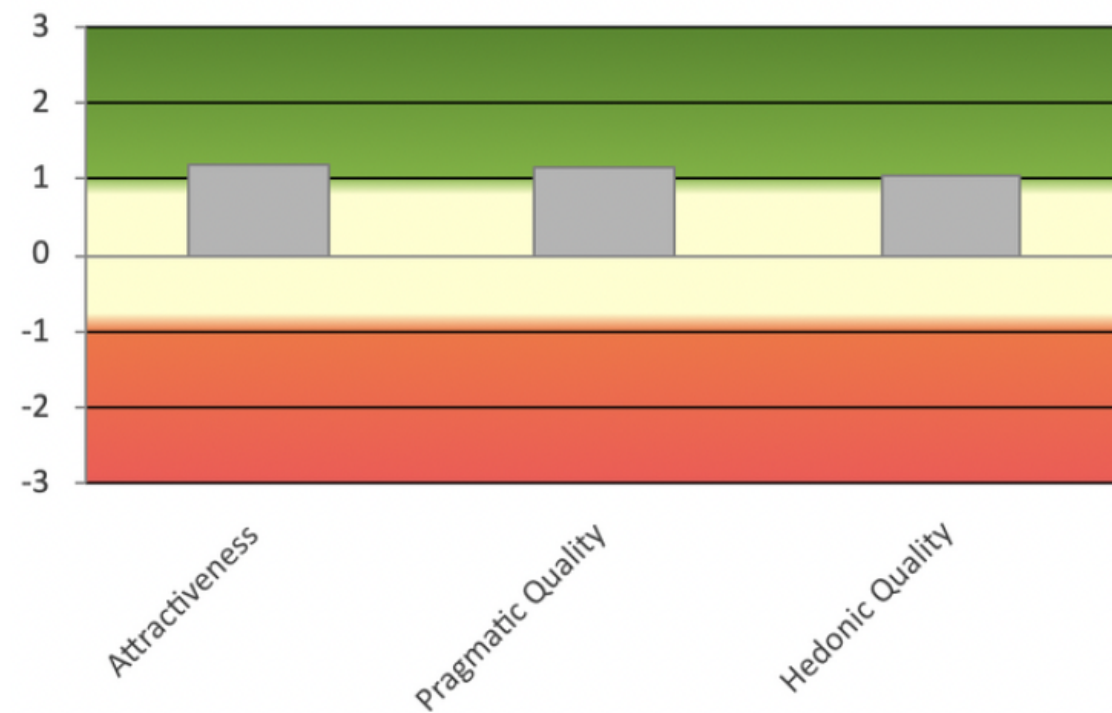


Figure 20. Aggregated UEQ scores displaying hedonic and pragmatic qualities as perceived by users.



Figure 21. Thematic grouping of qualitative user feedback from interviews and usability tests.

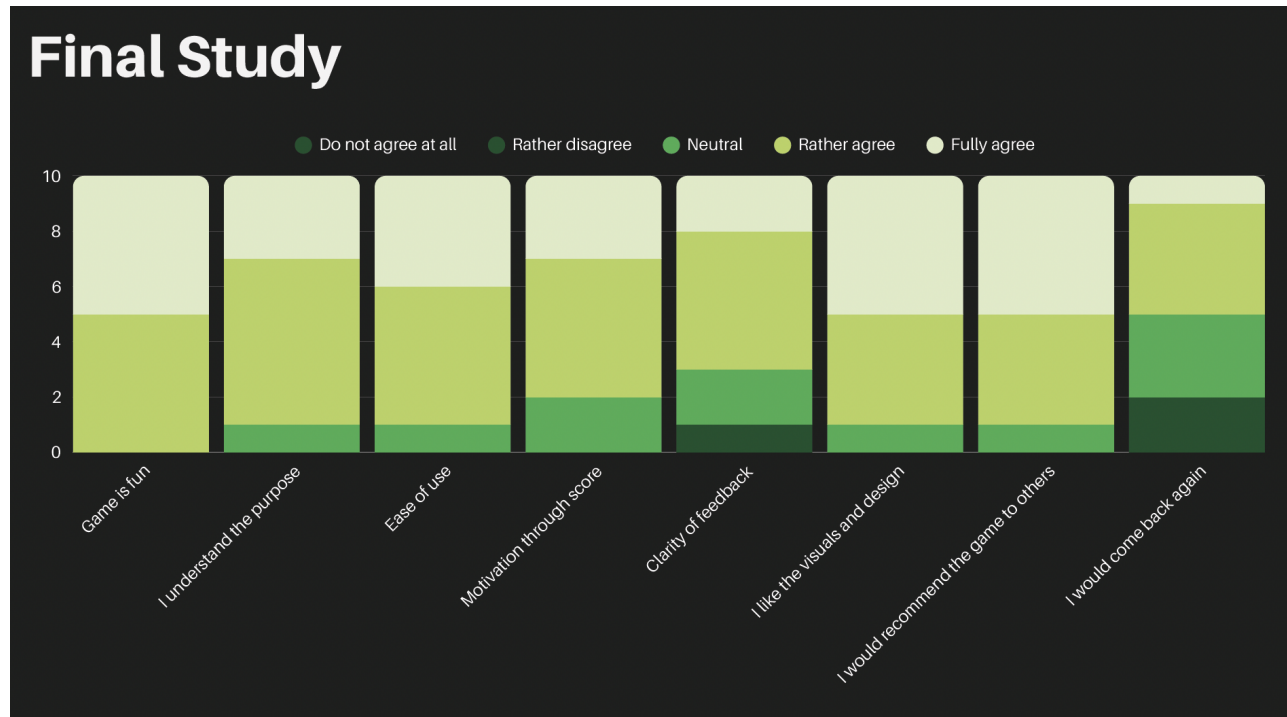


Figure 22. Final study results: distribution of agreement levels across fun, purpose, usability, feedback clarity, and willingness to reuse.